

Wilson Quilli

Professor Nalubandhu

SWENG 861: Software Construction

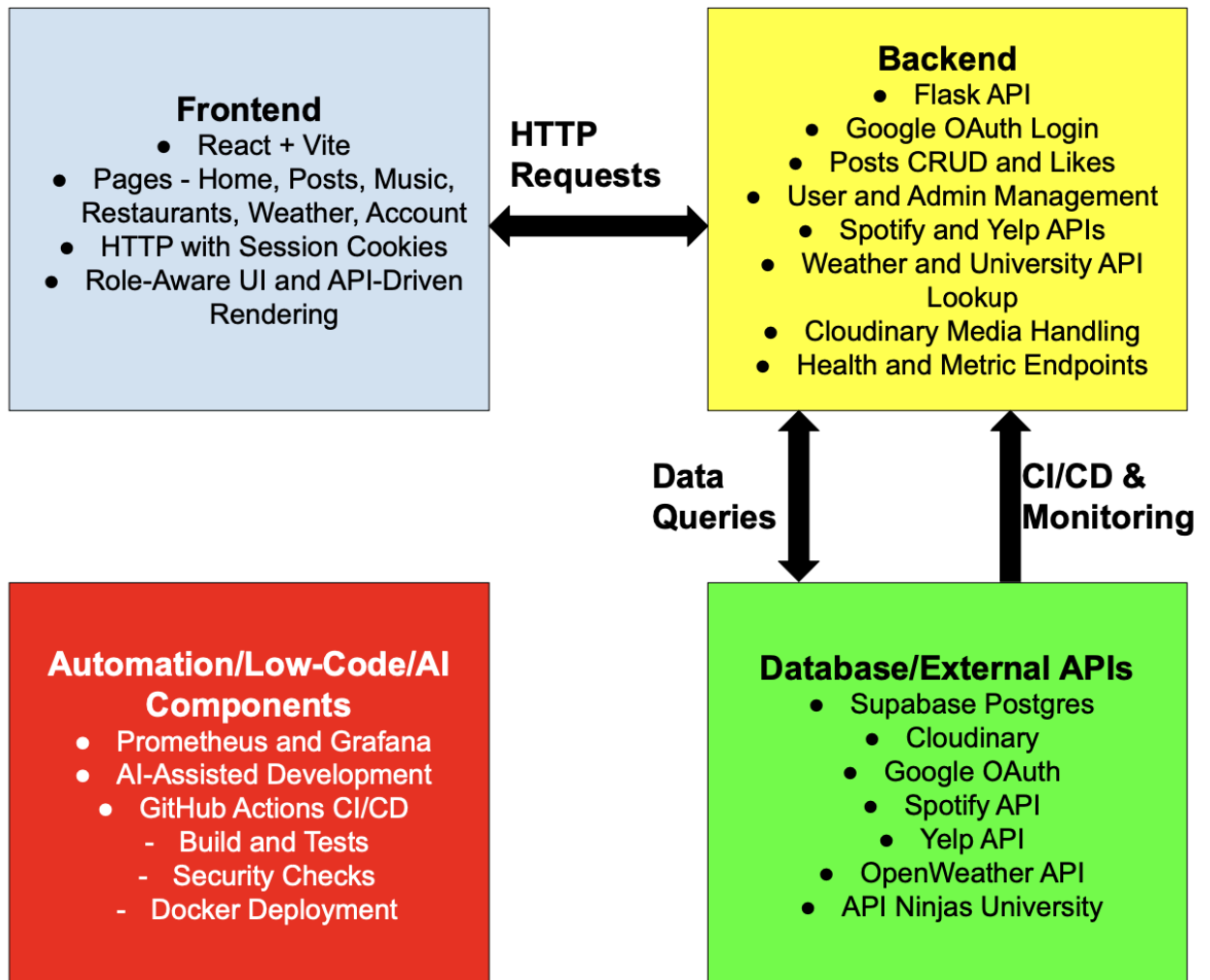
February 23rd, 2026

CollegeLife Documentation

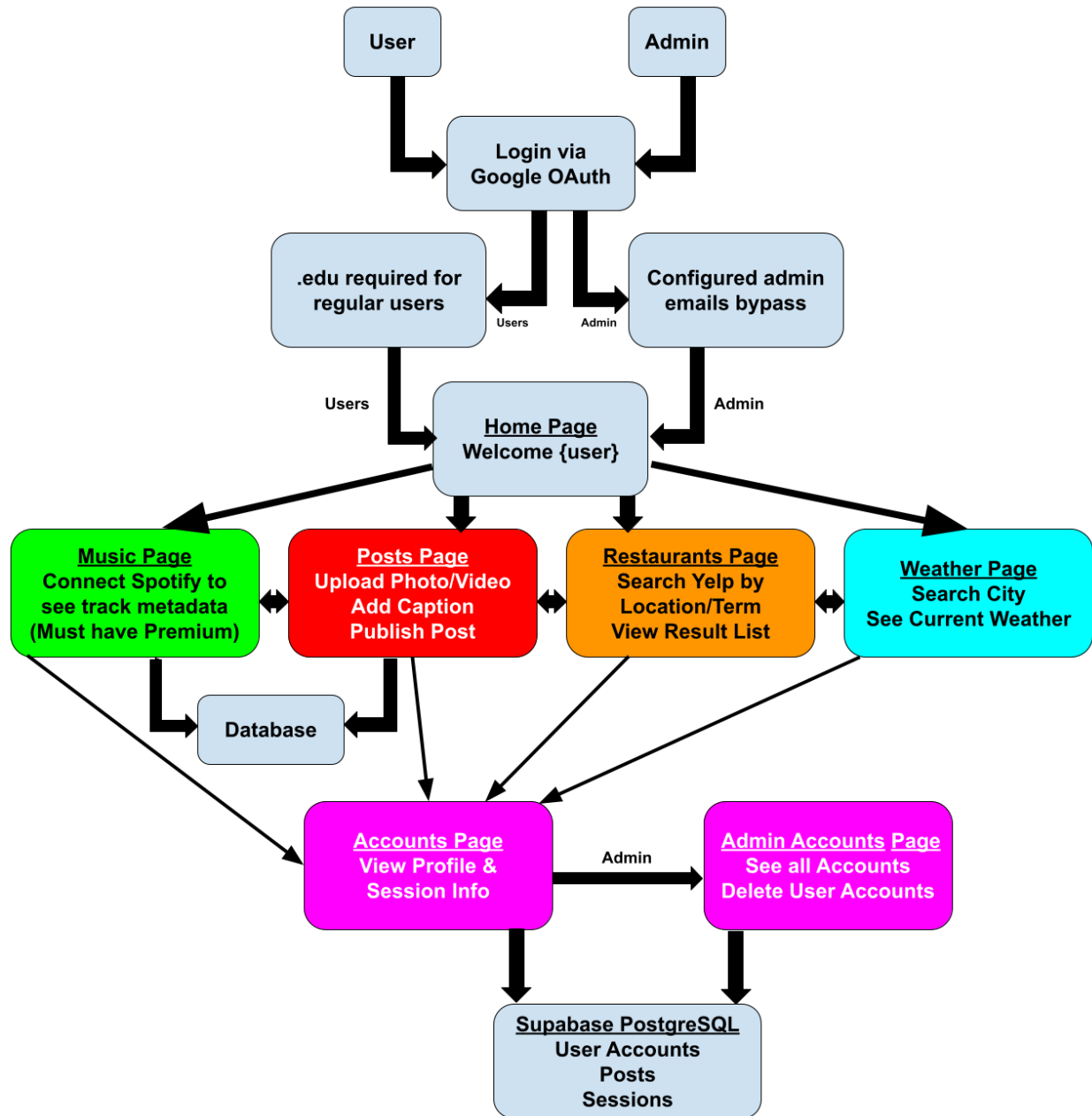
The goal of CollegeLife was to build a full-stack college-focused social web app where users can authenticate with Google, create and interact with media posts, and use music, restaurant, and weather features powered by external APIs. Success criteria included enforcing role-aware access (user vs admin), integrating Supabase and Cloudinary for data/media workflows, supporting Spotify/Yelp/Weather API-driven pages, and delivering a production-capable CI/CD and observability setup. The app also needed reliable CRUD behavior on posts, protected routes with correct auth responses, and deployment readiness on Render for backend and Vercel for frontend.

The system uses a React + Vite frontend and a Flask backend organized around route/controller logic plus shared business-rule helpers. Frontend pages call backend endpoints with credentialed requests; backend routes handle validation, authorization, persistence, and third-party API orchestration. Supabase is used as the primary data store for users/posts, while Cloudinary handles uploaded media assets. OAuth-based identity and session management are handled server-side through Authlib + Flask sessions. Operationally, the project includes structured logging, health/readiness endpoints, Prometheus metrics, and Grafana dashboard assets for runtime observability.

Simple Diagram of the Full-Stack CollegeLife Application



User & Admin Diagram



Key decisions included using Google OAuth for low-friction identity, enforcing admin behavior via configured admin emails, and implementing ownership rules so users can edit/delete only their own posts while admins can manage broader account operations. For deployment resilience, environment-driven URL configuration was prioritized to handle local and cloud host differences, and frontend API routing was made configurable via `VITE_API_BASE_URL`. CI was structured with explicit build, security, test, and package stages, with quality gates around tests and coverage. Additional hardening focused on OAuth redirect consistency, session/cookie behavior across origins, and robust API error handling for external integrations.

Testing is layered across backend and frontend: backend uses pytest for unit and API/integration tests with isolated fake/mock data behavior; frontend uses Vitest + React Testing Library for component/interaction/error-state validation; Playwright covers E2E user flows. The backend coverage gate is enforced in CI and currently exceeds the target threshold, and frontend tests/coverage are also wired into CI. Auth and authorization paths are explicitly tested (unauthenticated access, own-resource access, cross-user forbidden cases), and key UX flows such as login, post interactions, and account/admin behavior are covered with automated checks.

Locally, run backend (`python3 main.py`) and frontend (`npm run dev`) with correctly configured `.env` values and OAuth redirect URIs. In deployment mode, users can access the app directly through the Vercel URL while the backend runs on Render, with required env vars and OAuth callback settings configured in each provider.